# Fundamentals of Programming & Computer Science
# CS 15-112

## Lists

Dr. Hend Gedawy

# Practice- Alternating Sum

**Given a list of integers, calculate the alternating sum...**

[1, 2, 3, 4] => 1 - 2 + 3 - 4

# Practice- FindDuplicates

Write a function that takes a list and returns a list with all duplicates

# Last Lecture

- Lists Syntax
- Destruction
- Today:
  - Tuples
  - Aliasing
  - Practice

# Destruction

# Practice - ExamQ

6. (10 points) **Free Response:** destructiveListInList

Write the function `destructiveListInList(a, b, n)` which destructively modifies `a` (without modifying `b`) by adding all the values of `b` between elements `a[n-1]` and `a[n]` and returns the usual value returned by destructive functions. When `n == 0`, it adds all values of `b` onto the front of `a`. You may assume `0 <= n <= len(a)`. When `n == len(a)`, it adds all values of `b` at the back of `a` (the function behaves like `a.extend(b)`).

So this code works:

```
L = [3, 4, 1, 2]
destructiveListInList(L, [7, 6], 2)
assert(L == [3, 4, 7, 6, 1, 2])

L = [1, 3]
destructiveListInList(L, [4, 2], 2)
assert(L = [1, 3, 4, 2])

A = [4, 5]
B = [7, 8, 9]
destructiveListInList(A, B, 1)
assert(A == [4, 7, 8, 9, 5])
assert(B ==  [7, 8, 9])
```

You may not import or use any module other than copy. You may not use any method, function, or concept that we have not covered this semester. We may consider additional test cases not shown here.

# Aliasing

# CT Practice

```python
def f(L):
    L[0] = 15
    L.append(1)


def ct2(L):
    L = L + [1]
    M = L
    f(L)
    L += [1]
    if M is L:
        M[3] = 2
    print(L, M)


L = [0]
ct2(L)
print(L)
```

# Practice- Exam Q

```python
def f(a, b):
    b += ["hi"]
    a = b[:2]
    return a+b

def ct3(a):
    b = a
    c = b[:]
    b[0] = 42
    a.append("wow")
    b = f(a, b)
    if 42 in c:
        c.remove(42)
    print("a1",a)
    a[-2] = 112
    print("a2:", a)
    print("b2:", b)
    print("c2:", c)
z = ["feb", 17]
ct3(z)
print("z:", z)
```